

Welcome to Menu Territory!

Your mission is to help evaluate a model's ability to identify *all* issues in a Pull Request (PR) and provide helpful, actionable feedback (e.g., propose fixes, generate a final patch).

Project goals

By the end of this course, you will be able to:

- Create a **complex prompt** and **rubrics** that challenge cutting-edge AI models.
- Produce a **Golden Response** grounded in the provided **diff + base commits**.
- Evaluate whether rubrics successfully differentiate model quality (and stump at least one model).

What changed (new workflow)

- Repo + PR artifacts are provided in the task (no external harvesting).
- PR Harvest is removed (you do not locate PRs/commits yourself).
- Diff must be saved as a reference text in the **prompt** from the task artifacts.

Resources:

GUIDELINES: [Attempter Document](#)

Step 1: Read Task Specs & Constraints

The instructions in the tasks should take into account when the task is open: there is information about the PR link to work with, the particular GitHub repository, and the commits to take into consideration.

1) Confirm what's provided in the task

Review *all* artifacts included in the task (examples: repo link, issue title, base commit, PR commit, diff, test patch, etc.). You are not expected to search externally.

2) Confirm the “diff integrity”

Your diff must be used “as-is”:

*It must contain the *full diff*, including **all file changes**.*

The diff should match the relationship: $(\text{pr_commit} - \text{base_commit}) = \text{provided diff}$.

3) Important tip (what you may still do)

Use whatever workflow you prefer to inspect diffs/commits locally, but **do not do PR-hunting**—the task is already fully provisioned.

Task Artifacts Intake

> This module replaces "PR Harvest." You do **not** locate PRs anymore.

What to do instead

- Treat the task as the "source of truth."

Use only the **provided repo + provided PR context + provided diff.**

PR Diff

1. Copy the entire diff from the task artifacts section.
2. Paste the diff *exactly* as provided as a reference text in the prompt (no edits, no trimming).

Common failure modes

- * × Uploading an incomplete diff (missing files/hunks)
- * × Editing whitespace/lines in the diff "to clean it up"

Goal

Your prompt should instruct the model to act as a **senior software engineer** performing a PR review. The diff must contain issues worth catching.

- It should contain 2-3 sentences on the **Role + goal**

For example:

You are a senior software engineer reviewing a pull request in the qBittorrent repository. Your goal is to verify that the changes correctly implement the requested feature and that the implementation is production-ready and does not introduce regressions.

- It should contain around 3-8 sentences to establish the problem to pursue and the goal this PR wants to achieve.

Allowed issue types (examples)

- **Serious:** bug/correctness, security risk, missing tests, edge cases, perf regressions, API misuse
- **Non-serious:** formatting, dependency bumps, minor cleanup

Required prompt structure (copy/paste template)

Role + goal (2–3 sentences)

- “You are a senior software engineer reviewing a PR diff to verify it solves the problem statement, is production-ready, and introduces no regressions.”

Problem statement (3–8 lines)

- What's broken today?
- What behavior is expected?

Diff upload

- “A diff must be uploaded/pasted as a reference text in the prompt section. Review the diff as the sole source of code changes.”

Output requirements (pick ONE mode)

- **Review-only:** “Return a detailed PR review with issues, risks, and required changes before merge.”
- **Review + fix:** “If issues exist, propose a fix plan and provide a corrected patch.”

Diff upload

- “A diff must be uploaded/pasted as a reference text in the prompt section. Review the diff as the sole source of code changes.”

Output requirements (pick ONE mode)

- **Review-only**: “Return a detailed PR review with issues, risks, and required changes before merge.”
- **Review + fix**: “If issues exist, propose a fix plan and provide a corrected patch.”

Prompt quality checklist

- Persona is explicit (senior engineer / code reviewer).
- Problem statement is concrete and testable.
- Diff is uploaded/pasted as a reference text.
- Output mode is unambiguous (review-only vs review+fix).

Craft your prompt

Write

Side-by-side

Full Preview

H1H2H3

B*I*

“”</>

Write your markdown here...

T

Add reference text



Next

Step 3: Generate Golden Response based on PR

What “Golden Response” means

This is the ideal senior-engineer PR review comment: accurate, actionable, and aligned to the prompt’s output mode. It must not contain objective inaccuracies.

Required structure (use these headings)

1. Summary (2–4 sentences)

- * Does the patch solve the problem?
- * Top 1–2 issues that matter most.

2. Blockers (must-fix before merge)

For each blocker, include:

- What’s wrong
- Why it matters
- Exact fix
- Where (file path + line range if available)

3. Non-blocking suggestions

- Keep short; avoid scope creep.

4. Tests / Validation

- Specific edge cases to test
- What should fail before and pass after
- Require regression tests for behavior changes
- Unit test files shouldn't be included here, only the test cases

5. Fix plan *(ONLY if review+fix)*

- 3–6 steps max
- Each step maps to a blocker

6. Corrected patch *(ONLY if review+fix)*

- Provide a corrected patch file
- Fix all blockers; keep changes minimal (no refactors)

Step 4: Generate Rubrics based on the golden response

Minimum requirements

Create **at least 15 rubric items** spanning:

1. Core correctness (must-catch issue)
2. Required fix correctness
3. Regression/edge-case coverage
4. Scope discipline
5. Clarity/actionability
6. Hygiene checks when applicable

Rubrics must be self-contained

Ask: "Using only the prompt, the rubrics, and the model response, can I grade this reliably?" If not, rewrite the rubric.

Coverage requirement

Every serious issue in the PR should be covered by at least one rubric item.

Rubric Types

- *Instruction Following*

Ensures the response adheres to explicit directions in the prompt (format, constraints, language, libraries, required elements).

- *Bug Discovery*

Verifies the model identifies and suggests solutions for bugs or issues present in the PR.

- *Logic Reasoning*

Verifies the model identifies and suggests solutions for any reasoning flaws.

- *Style and Tone*

NICE TO HAVE ONLY - Checks for any formatting or tone requirements that would make the response better looking.

Weighting rules

- Each rubric criterion must have **one** of the following weights:
 - 5 — Mandatory** This is a core requirement for adequately answering the prompt, is clearly expected based on the prompt, and it would be hard to envision an acceptable response without it.
 - 3 — Important** This makes the response substantially better and is reasonably or implicitly expected based on the prompt, though you can envision an acceptable response without it as long as all other criteria are met.
 - 1 — Nice to have (Optional)** This is a nice-to-have and necessary for a perfect response, but a response can still be strong without it.

Write criteria that encompass all requirements needed to fulfill this prompt

1/15 completed

1

The response includes an...

Category *

If unsure, reference the category definitions.

Options [View details](#)

☐ Instruction Following

☐ Bug Discovery

☐ Logic Reasoning

☐ Style and Tone

How Does This Criterion Affect The Response? *

Options [View details](#)

☒ Mandatory

☐ Important

☐ Nice to Have

Weight (Mandatory) *

This is a core requirement for adequately answering the prompt, is clearly expected based on the prompt, and it would be hard to envision an acceptable response without it.

5

Rubrics - Guidelines

There are several requirements for the rubric:

1. Each criteria **HAS** to begin with "The model should..."
2. There should be a **MINIMUM** of 5 criteria (1 for correctness, 1 for format adherence, and the rest for reasoning quality)
3. Please add as many criteria as needed to cover all **crucial** reasoning steps to derive the final answer
4. Appropriately **atomic** (related to a single challenge in the prompt and avoids stacking multiple expectations into one criterion)
5. **Specific** (containing sufficient visual/textual detail and is always a "closed-ended" list: no using "such as" or "examples")
6. **Self-contained** (can be evaluated without any knowledge or information other than what is contained in that **singular** criteria)
7. **Accurate** (correct by fact check/reason)
8. Is not **Redundant** (multiple criteria should not test for the same thing)
9. Criterion **must not** be open-ended, they need to be definitive

Step 5: Evaluate the Model Evaluation based on Rubrics

Pass/fail gate for task difficulty

Proceed only if:

- *At least one model scores **below 70%** , AND*
- *The **average** of the three models is **below 70%***

If it doesn't meet the gate

Increase difficulty by:

- Tightening rubrics (more specific "must-catch" items)
- Clarifying prompt requirements
- Ensuring the diff actually contains subtle but real issues that are gradeable

Code Review

What is the rate range for rubrics that are labeled as **Important**?

Attempts remaining: 0

2

☐

3

☒

1

☐

7

☐

✓ This is correct

How many sentences should be minimally added for the Role + Goal in the Prompt?

Attempts remaining: 0

1

☐

4

☐

5

☐

3

☒

✓ This is correct

What features should **NOT** be included in the **Golden Response**?

Attempts remaining: 0

Test/Validation

☐

Non-blocking suggestion

☐

Fix plan

☐

Unit test file

☒

✓ This is correct

How many rubric items **minimum** are required?

Attempts remaining: 0

15

☒

2

☐

10

☐

12

☐

✓ This is correct

The rubric must be written in fewer than 15 items so it's easy to apply

☐

The rubric can be graded using only the prompt, the rubric, and the model response. No external context needed

☒

The rubric must reference the exact line numbers from the repository so graders can verify quickly.

☐

The rubric must include screenshots of the PR diff to be considered complete.

☐

✓ This is correct

Which rubric weight corresponds to "core requirement"?

Attempts remaining: 0

Weight 1

☐

Weight 3

☐

Weight 5

☒

Weight 0

☐

✓ This is correct

Congratulations!

You can now produce high-quality Menu Territory tasks that:

- *Use only **task-provided artifacts*** (no PR hunting).
- *Correctly handle the **diff** as an **uploaded reference text in the prompt in taxonomy**.*
- *Generate a **Golden Response*** with actionable blockers and test coverage.

*Build **self-contained, weighted rubrics*** that actually separate model performance.

Final submission checklist

- Prompt includes persona + problem + output mode
- Diff file uploaded in taxonomy as a reference text in the prompt
- Golden Response matches required structure and task mode
- ≥ 15 rubrics, weighted, self-contained, cover all serious issues
- Evaluation gate satisfied (<70% condition)

Coming up:

- Screening